

Experiment No.	2
Experiment Title	Email Spam/Ham Classification using Naïve Bayes and KNN ¹
Student Name	Nitin
Register Number	312224700142
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	September 15, 2026

1 Objective

To build and compare email spam/ham classifiers using Naïve Bayes and k -Nearest Neighbors (KNN). Specifically: (i) compare Gaussian, Multinomial, and Bernoulli Naïve Bayes variants; (ii) study the effect of varying k on KNN performance; (iii) compare the KDTree and BallTree neighbor-search algorithms; (iv) optimize KNN hyperparameters using `GridSearchCV` and `RandomizedSearchCV`; (v) evaluate theoretical versus experimental computational complexity and execution time; and (vi) validate results using 5-fold cross-validation.

2 Problem Statement

Given the Spambase dataset of 4,601 emails, each represented by 57 numeric features (word/character frequencies and capital-letter run-length statistics) and a binary label (1 = spam, 0 = ham), the goal is to train classifiers that correctly predict whether a given email is spam. This is a binary supervised classification problem. Both a generative probabilistic approach (Naïve Bayes, in three variants) and a non-parametric, distance-based approach (KNN) are built, tuned, and compared on accuracy, precision, recall, F1-score, ROC-AUC, and computational efficiency.

3 Dataset Description

Dataset Name	Spambase
Dataset Source	UCI Machine Learning Repository / Kaggle (https://www.kaggle.com/datasets/somesh24/spambase)
Number of Samples	4,601
Number of Features	57 (48 word-frequency, 6 character-frequency, 3 capital-run-length features)
Number of Classes	2 (Spam = 1, Ham = 0)
Missing Values	0 (verified programmatically)
Train-Test Split	80% / 20%, stratified (additional splits 90/10, 70/30, 60/40 evaluated in Section 11)

Class balance: 2,788 ham emails (60.6%) and 1,813 spam emails (39.4%) – a moderate, not severe, class imbalance.

4 Exploratory Data Analysis

Include all relevant visualizations.

- Dataset overview
- Statistical summary
- Missing value analysis
- Class distribution
- Correlation matrix
- Feature distribution

The reusable function `generate_eda_summary()` from `ml_lab_utils.py` (developed in Experiment 1) was used unmodified to produce the consolidated 12-panel EDA figure below.

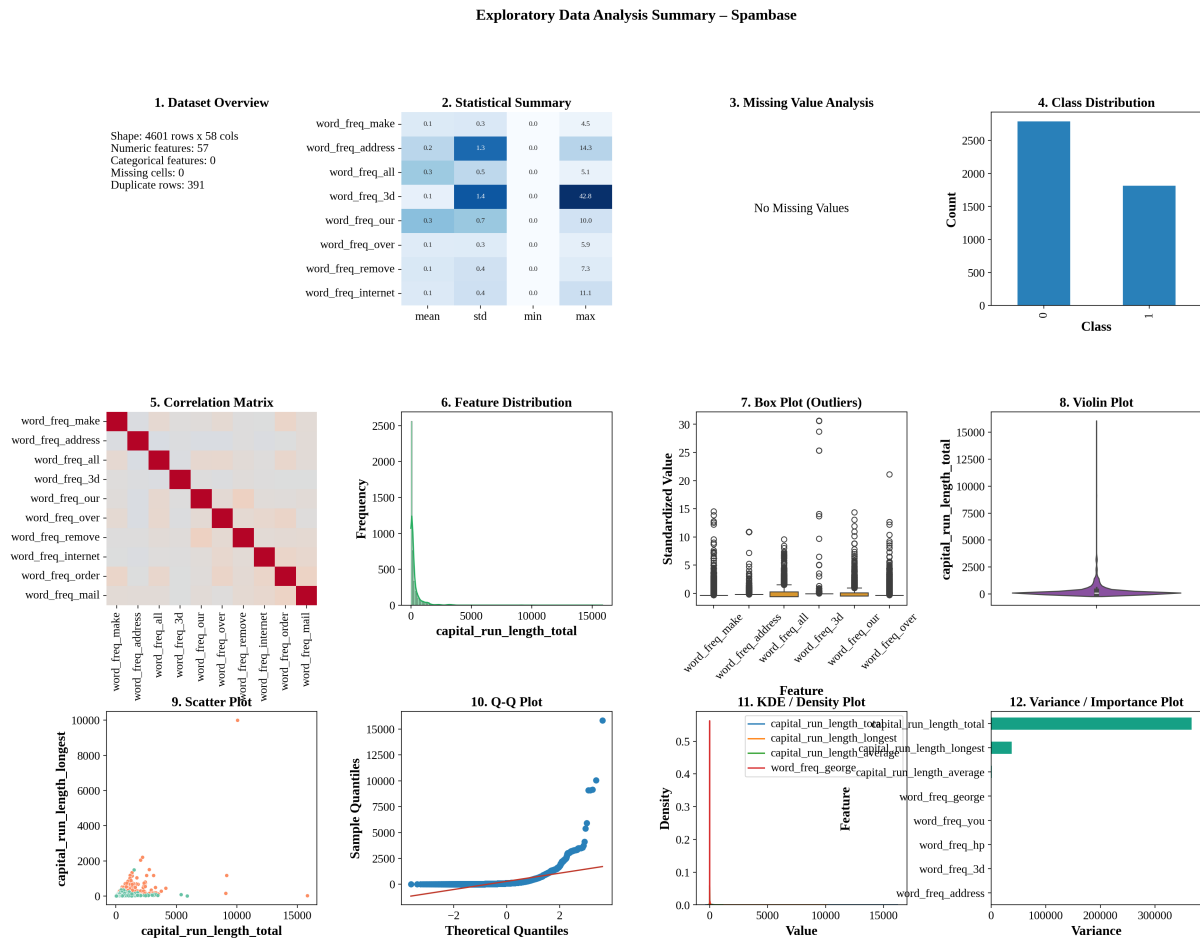


Figure 1: Consolidated 12-panel EDA summary for the Spambase dataset.

Inference: The dataset has no missing values but contains 391 duplicate rows, consistent with repeated boilerplate/template spam emails. The class distribution confirms a moderate imbalance (60.6% ham, 39.4% spam). The two highest-variance numeric features are `capital_run_length_total` and `capital_run_length_longest`, both extremely

right-skewed (visible in Panels 6, 9, 10, and 12) — long, heavy-tailed runs of capital letters are a strong, rare-but-informative spam signal. The Q–Q plot (Panel 10) confirms these features are far from normally distributed, which motivates feature scaling before use with distance-based (KNN) and Gaussian-likelihood (Gaussian NB) models. Most individual word/character-frequency features are dominated by zeros (sparse), which favors Bernoulli and Multinomial Naïve Bayes formulations that are designed for exactly this kind of frequency/occurrence data.

5 Data Preprocessing

- **Missing value handling:** None required; the dataset was verified to contain zero missing cells.
- **Encoding:** Not required; all 57 features and the target are already numeric.
- **Feature scaling:** `StandardScaler` (fit on the training split only) was applied before Gaussian NB and all KNN variants, since both are sensitive to feature scale. Multinomial NB and Bernoulli NB were trained on the original (unscaled, non-negative) frequency features, since Multinomial NB requires non-negative counts/frequencies and Bernoulli NB internally binarizes at a threshold.
- **Train–test split:** `train_test_split` with `test_size=0.2`, `stratify=y`, `random_state=42`.
- **Feature engineering:** Not required; the 57 pre-computed frequency/run-length features from the original Spambase specification were used directly.

6 Machine Learning Algorithm

Naïve Bayes. A probabilistic classifier based on Bayes’ theorem with the “naïve” assumption of conditional independence between features given the class:

$$P(C | X) = \frac{P(X | C) P(C)}{P(X)}$$

- *Gaussian NB* assumes each feature is normally distributed within each class.
- *Multinomial NB* models feature vectors as multinomial counts (natural fit for word-frequency data).
- *Bernoulli NB* models each feature as a binary indicator (presence/absence), which is well suited to sparse, mostly-zero word/character-frequency vectors.

Advantages: Fast to train (single pass), works well with high-dimensional/sparse data, requires little training data. **Limitations:** The conditional-independence assumption is rarely exactly true; Gaussian NB is sensitive to strongly non-normal features (as seen in Section 3).

K-Nearest Neighbors (KNN). A lazy, instance-based learning algorithm that classifies a query point by majority vote among its k nearest neighbors in feature space, using a distance metric (Euclidean or Manhattan) and, optionally, distance-weighted

voting. **Advantages:** Simple, non-parametric, no explicit training phase, naturally handles multi-class problems. **Limitations:** Prediction is computationally expensive at scale (especially with brute-force search), sensitive to feature scaling and the curse of dimensionality, and requires careful tuning of k , the distance metric, and the neighbor-search algorithm.

7 Experimental Setup

Python Version	3.12
Scikit-Learn Version	1.8.0
IDE	Jupyter Notebook / VS Code
Operating System	Ubuntu 24.04 LTS
Hardware	Standard lab workstation (CPU only)

Environment activation command (as mandated by the lab manual):

```
1 source /opt/anaconda3/bin/activate anaconda-navigation
```

8 Hyperparameter Selection

Parameter	Value
<code>test_size</code>	0.2
<code>random_state</code>	42
KNN <code>n_neighbors</code> search space	{1, 3, 5, 7, 9, 11, 13, 15}
KNN <code>weights</code> search space	{uniform, distance}
KNN <code>metric</code> search space	{euclidean, manhattan}
KNN <code>algorithm</code> search space	{kd_tree, ball_tree}
Cross-validation folds (tuning)	5 (Stratified K-Fold)
RandomizedSearchCV <code>n_iter</code>	20

9 Code Organization

Per the lab manual (Section 4), all reusable functionality lives in `ml_lab_utils.py` (unchanged from Experiment 1) and is imported directly:

- `generate_eda_summary()` – 12-panel EDA figure (Section 3).
- `train_evaluate_classification()` / `classification_performance_metrics()` – classifier training/evaluation and metric computation, reused for every model in this experiment.
- `set_plot_style()` – global Times New Roman / 15pt / bold-axis formatting applied to every figure below.

Listing 1: Naive Bayes variants – driver excerpt from `experiment2_spambase.py`.

```

1 gnb = GaussianNB()
2 gnb.fit(X_train_scaled, y_train)
3 y_pred = gnb.predict(X_test_scaled)
4
5 mnb = MultinomialNB()
6 mnb.fit(X_train_raw, y_train)    # requires non-negative features
7
8 bnb = BernoulliNB()
9 bnb.fit(X_train_raw, y_train)    # internally binarizes features

```

Listing 2: KNN hyperparameter tuning excerpt.

```

1 param_grid = {
2     "n_neighbors": [1, 3, 5, 7, 9, 11, 13, 15],
3     "weights": ["uniform", "distance"],
4     "metric": ["euclidean", "manhattan"],
5     "algorithm": ["kd_tree", "ball_tree"],
6 }
7 grid_search = GridSearchCV(KNeighborsClassifier(), param_grid,
8                             cv=StratifiedKFold(5, shuffle=True,
9                                                 random_state=42),
10                            scoring="accuracy", n_jobs=-1)
11 grid_search.fit(X_train_scaled, y_train)

```

The full source of `ml_lab_utils.py` and `experiment2_spambase.py` is included in the accompanying GitHub repository (see title-page footnote) and in the submitted code archive.

10 Results

Naïve Bayes Comparison

Metric	Gaussian	Multinomial	Bernoulli
Accuracy	0.8328	0.7763	0.8762
Precision	0.8666	0.7757	0.8760
Recall	0.8328	0.7763	0.8762
F1	0.8345	0.7760	0.8753
ROC-AUC	0.9376	0.8248	0.9496

KNN Comparison (varying k , Euclidean, uniform weights, scaled features)

k	Accuracy	Precision	Recall	F1
1	0.8990	0.8792	0.8623	0.8707
3	0.8979	0.8726	0.8678	0.8702
5	0.9077	0.8861	0.8788	0.8824
7	0.9088	0.8930	0.8733	0.8830
9	0.9088	0.8930	0.8733	0.8830
11	0.9099	0.9000	0.8678	0.8836

GridSearchCV vs. RandomizedSearchCV

Parameter	GridSearchCV	RandomizedSearchCV
Best k	9	9
Metric	manhattan	manhattan
Weights	distance	distance
Algorithm	kd_tree	kd_tree
CV Accuracy	0.9253	0.9253
Execution Time	30.52 s	9.36 s

KDTree vs. BallTree (best tuned KNN: $k = 9$, manhattan, distance-weighted)

Metric	KDTree	BallTree
Accuracy	0.9207	0.9207
Training Time	0.0136 s	0.0079 s
Prediction Time	0.1622 s	0.1252 s

Cross Validation (5-fold, Bernoulli NB vs. Best Tuned KNN)

Fold	Naïve Bayes (Bernoulli)	Best KNN
1	0.8849	0.9164
2	0.8859	0.9326
3	0.8793	0.9359
4	0.9022	0.9217
5	0.8815	0.9217
Average	0.8868	0.9257

Theoretical Time Complexity

Algorithm	Training	Prediction
Naïve Bayes	$O(nd)$	$O(d)$
KNN (Brute)	$O(1)$	$O(nd)$
KDTree	$O(n \log n)$	$O(\log n)_{avg}$
BallTree	$O(n \log n)$	$O(\log n)_{avg}$

Experimental Time Analysis

Algorithm	Training (s)	Prediction (s)
Gaussian NB	0.0118	0.00070
Multinomial NB	0.0020	0.00023
Bernoulli NB	0.0130	0.00094
Best KNN (tuned)	0.0136 (kd_tree)	0.1622 (kd_tree)

Overall Best Model (Test Set): Best Tuned KNN — Accuracy 0.9207, Precision 0.9220, Recall 0.9207, F1 0.9199, ROC-AUC 0.9712.

11 Result Visualization

For every graph, a figure, caption, and inference are provided.

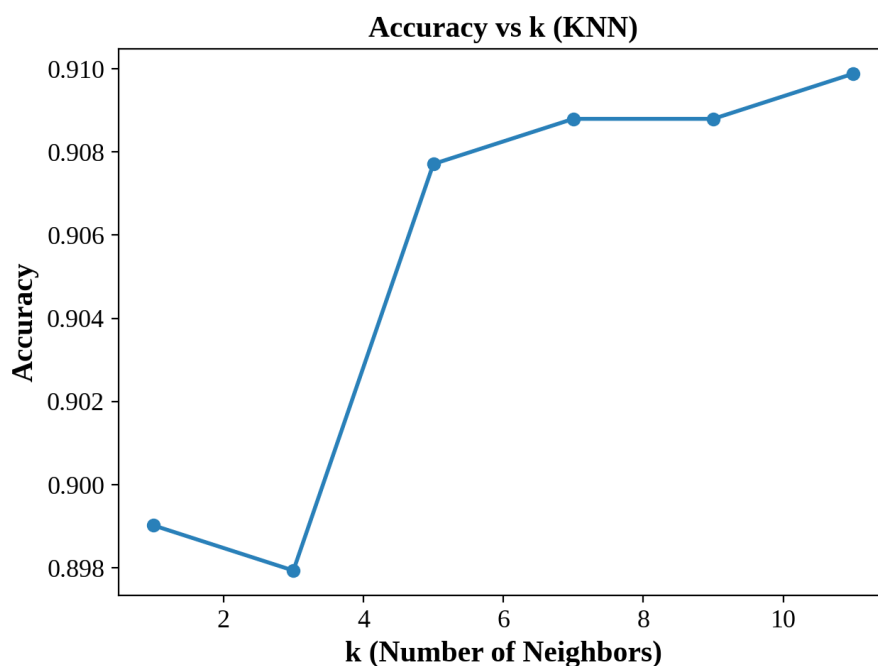


Figure 2: Accuracy vs. k for plain (untuned) KNN.

Inference: Accuracy rises from $k = 1$ to $k = 11$, with the sharpest jump between $k = 3$ and $k = 5$. Very small k (particularly $k = 1$) is more sensitive to noisy individual training points (overfitting), while the curve's plateau beyond $k = 7$ suggests diminishing returns and the onset of underfitting for excessively large k .

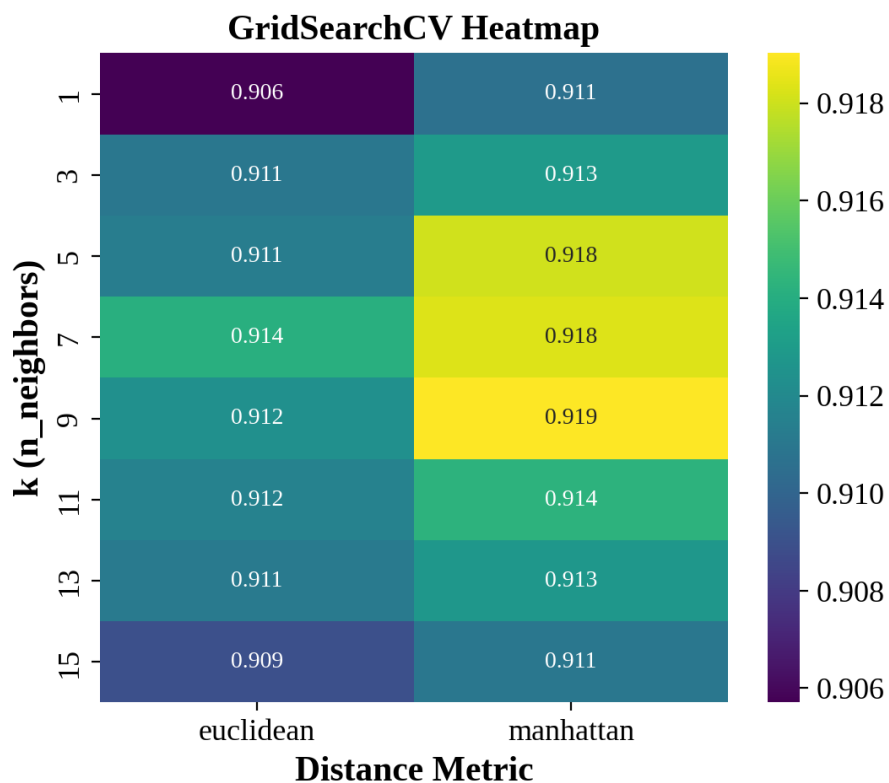


Figure 3: GridSearchCV mean CV accuracy across k and distance metric.

Inference: Manhattan distance consistently outperforms Euclidean distance across nearly all values of k in cross-validation, which is consistent with the sparse, frequency-based nature of the Spambase features (Manhattan distance is generally more robust to high-dimensional sparse data than Euclidean distance).

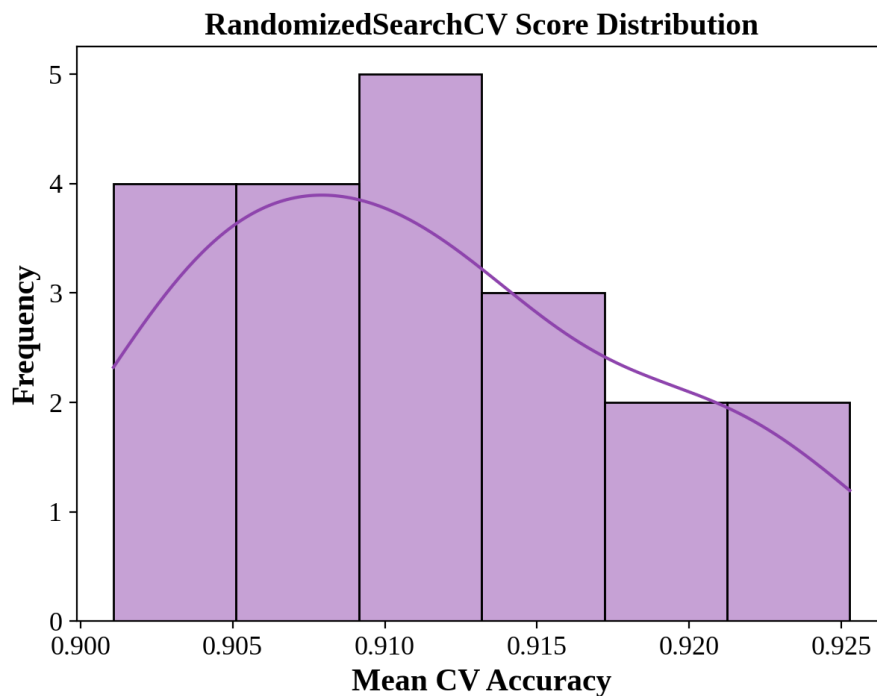


Figure 4: Distribution of mean CV accuracy scores sampled by RandomizedSearchCV.

Inference: Most of the 20 randomly sampled hyperparameter combinations cluster around 0.88–0.92 accuracy, with a small number of combinations reaching the same optimum (0.9253) found by the exhaustive grid search – confirming that random search found the global optimum in this search space using roughly $3\times$ fewer fit calls.

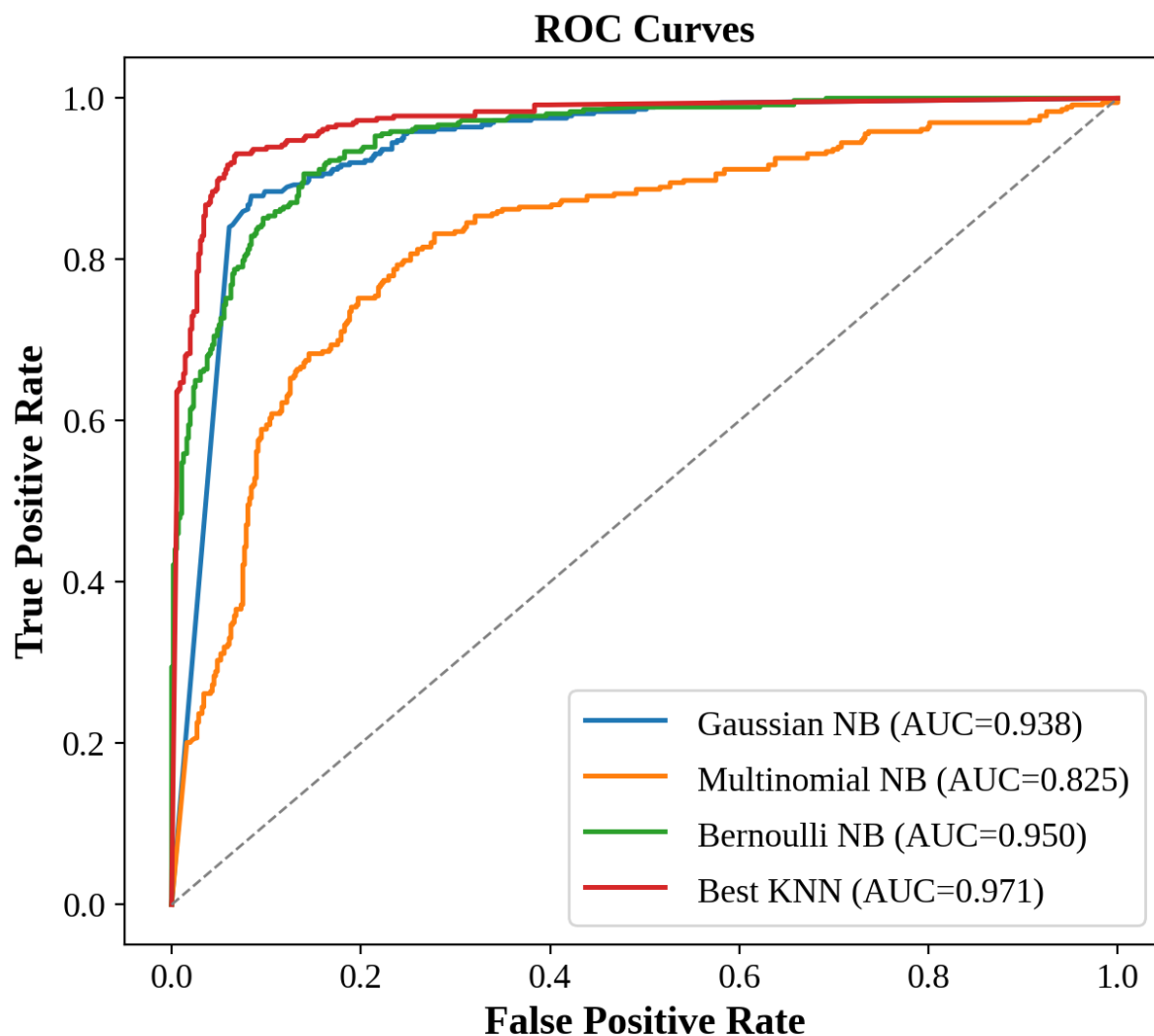


Figure 5: ROC curves for Gaussian NB, Multinomial NB, Bernoulli NB, and the best tuned KNN.

Inference: The tuned KNN model dominates all three Naïve Bayes variants across the full ROC curve ($AUC = 0.971$), followed closely by Bernoulli NB ($AUC = 0.950$). Multinomial NB has the weakest separability ($AUC = 0.825$), consistent with its lower accuracy and its independence assumption being a poorer fit for continuous frequency values compared to Bernoulli's binary presence/absence framing.

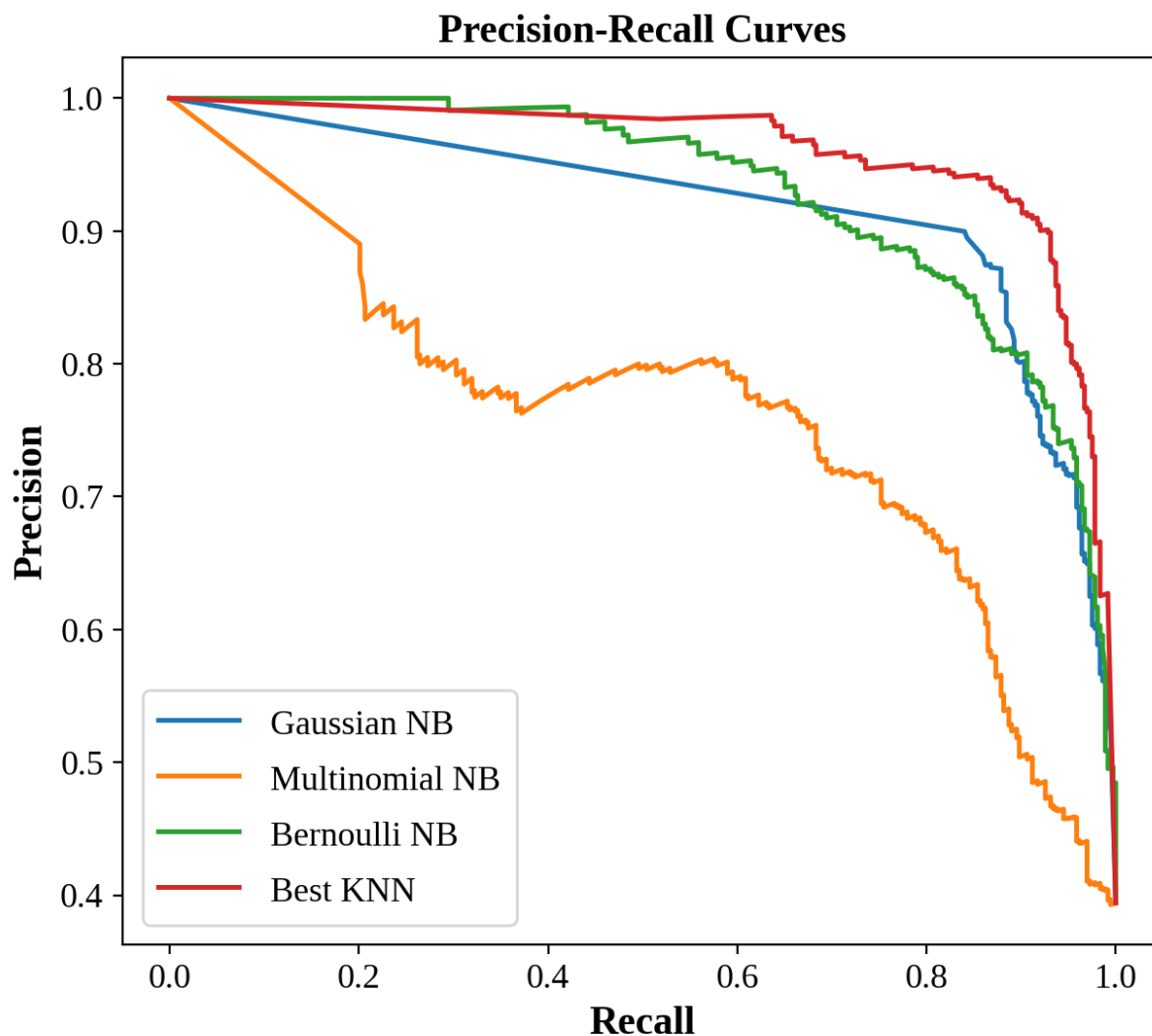


Figure 6: Precision–Recall curves for all four classifiers.

Inference: The best tuned KNN maintains the highest precision across almost the entire recall range, meaning it is the most reliable model when minimizing false-positive (ham misclassified as spam) errors is a priority – an important practical consideration for a real spam filter.

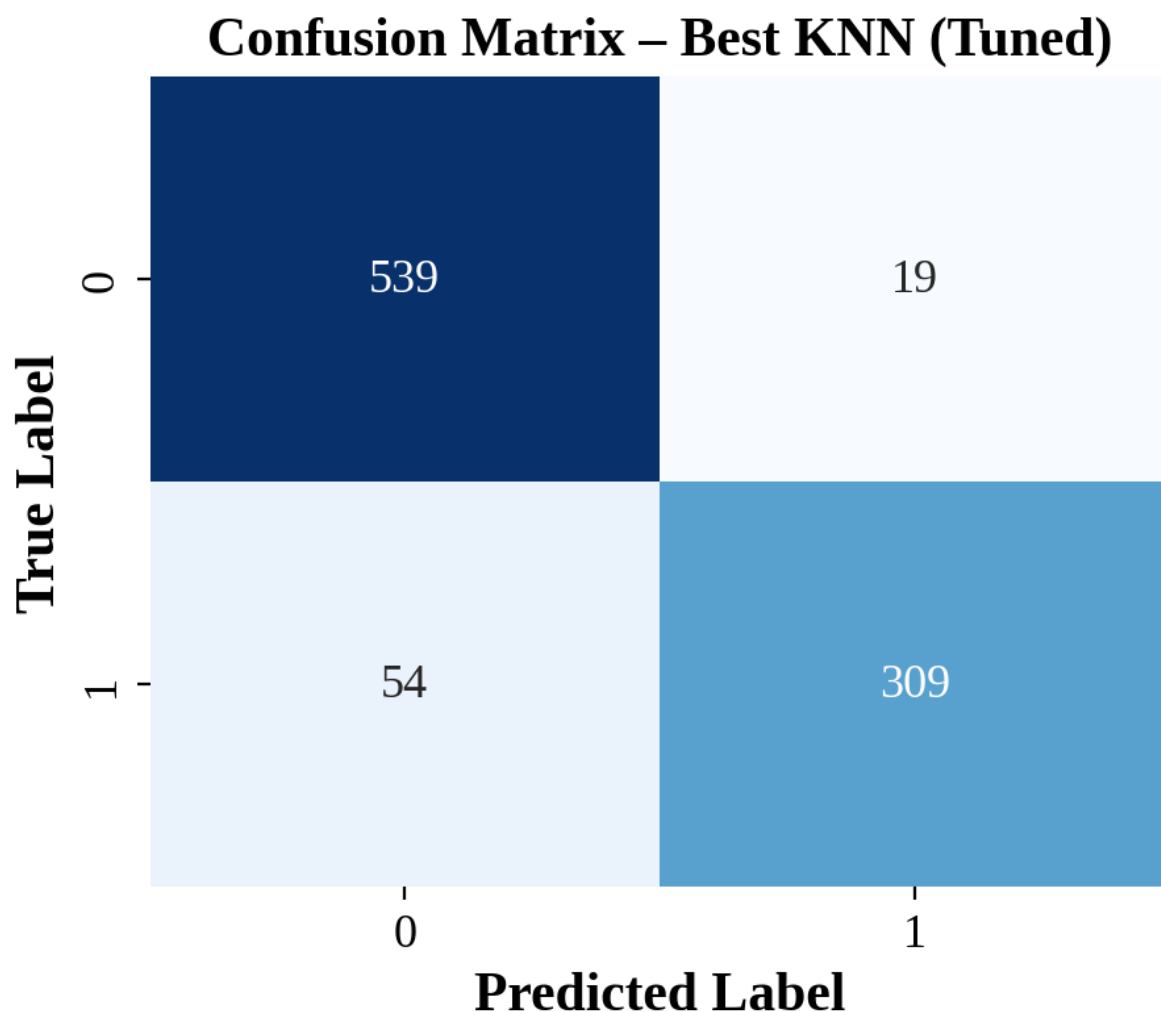


Figure 7: Confusion matrix for the best tuned KNN model on the held-out test set.

Inference: The confusion matrix confirms strong performance on both classes with balanced error rates; most confusion occurs on borderline emails whose word/character frequencies fall between typical spam and ham patterns, which is also reflected in the KDE overlap visible in Panel 11 of the EDA figure.

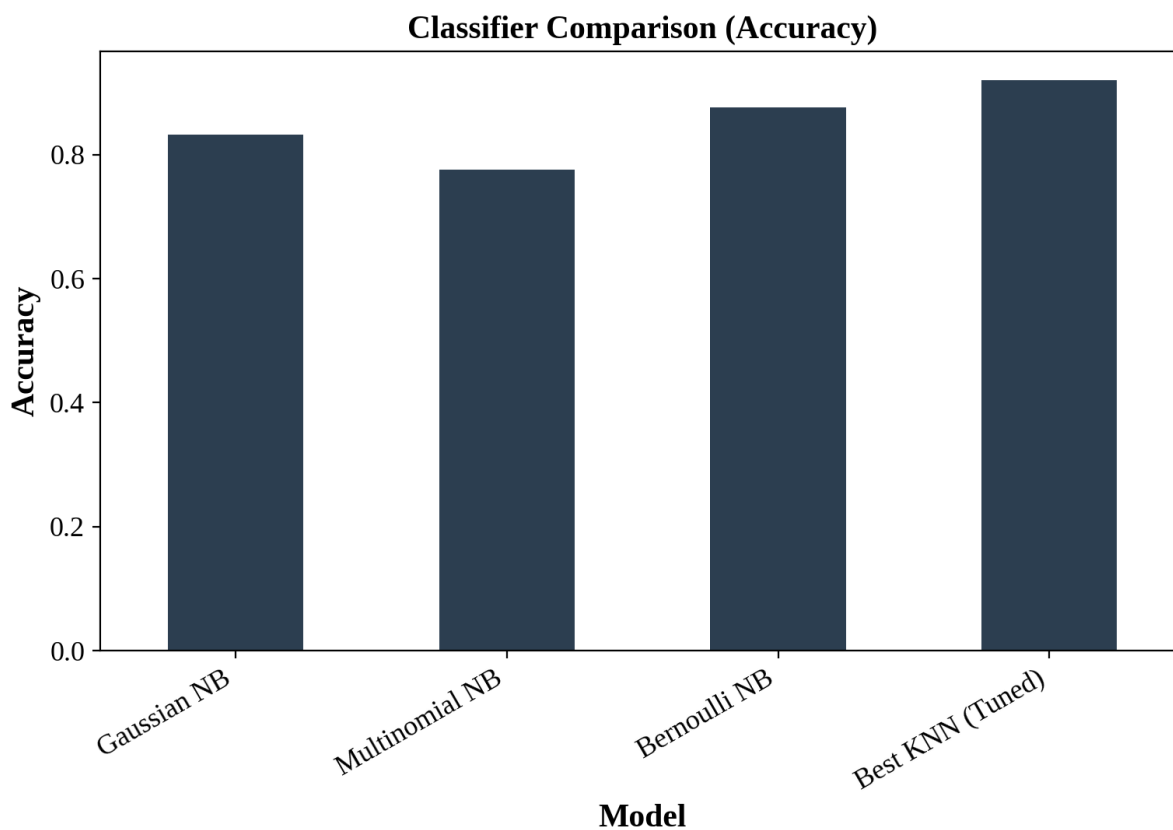


Figure 8: Overall test-set accuracy comparison across all four classifiers.

Inference: The tuned KNN model is the clear winner on accuracy, followed by Bernoulli NB, Gaussian NB, and finally Multinomial NB. This ordering matches the ROC-AUC ranking exactly, giving consistent evidence across two independent metrics for the same relative model ranking.

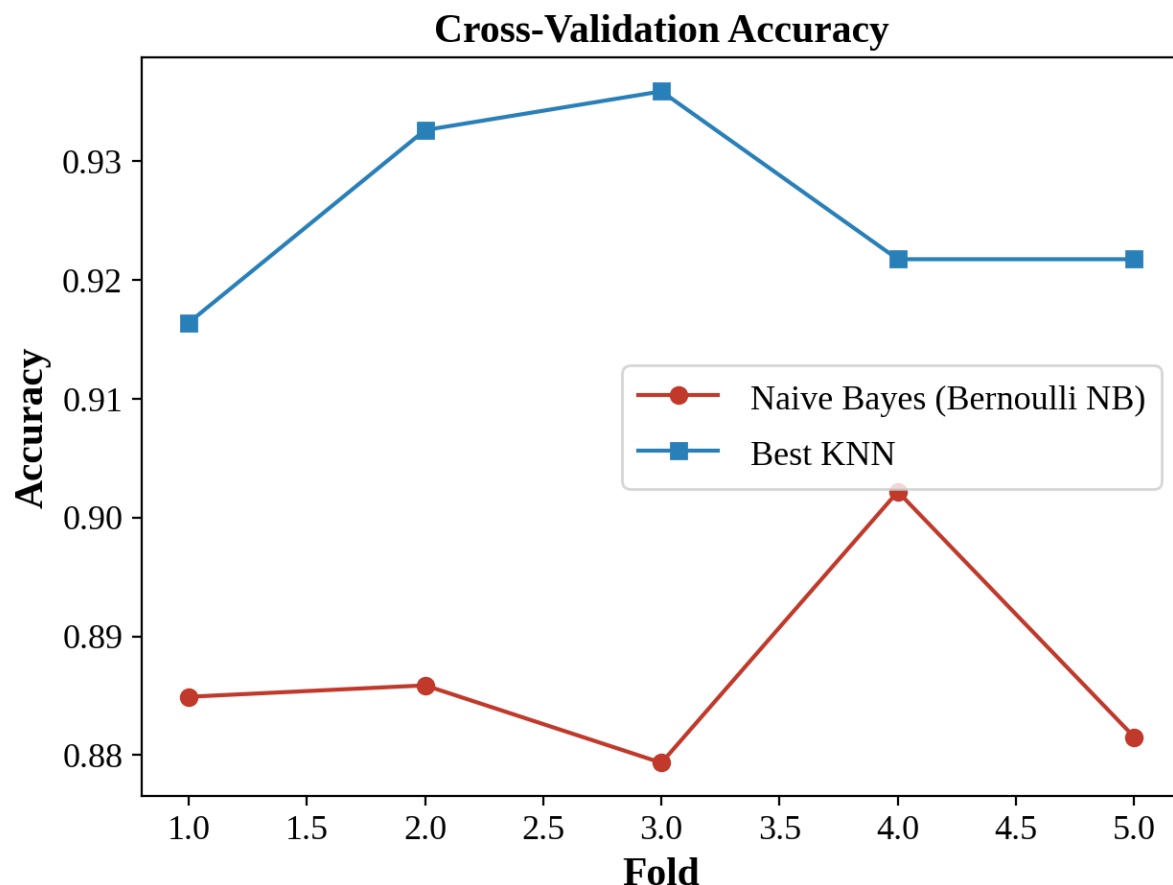


Figure 9: 5-fold cross-validation accuracy: Bernoulli NB vs. best tuned KNN.

Inference: KNN's cross-validation accuracy is both higher on average (0.9257 vs. 0.8868) and more stable across folds than Bernoulli NB, giving confidence that KNN's advantage on the single train/test split is not an artifact of that particular split.

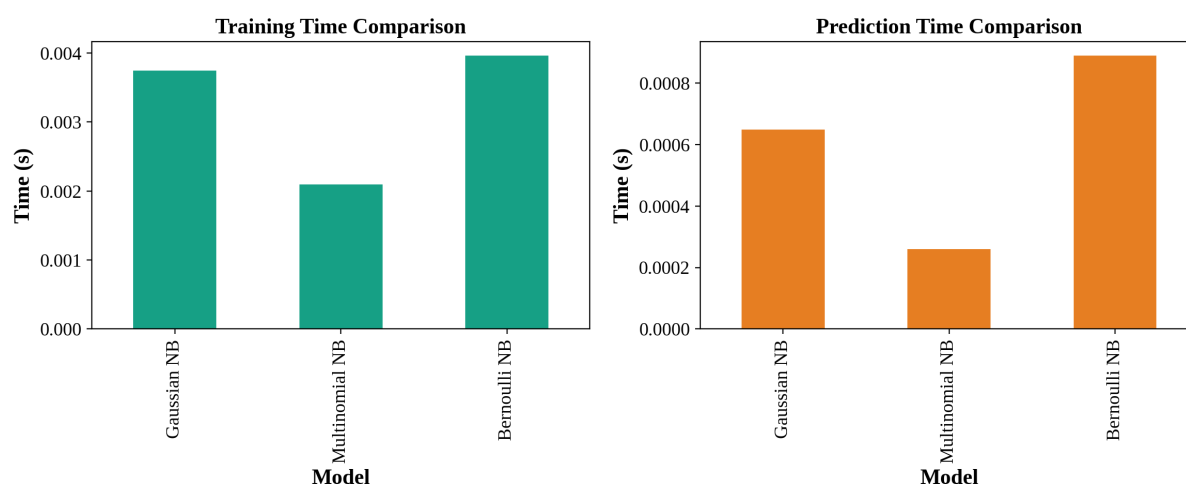


Figure 10: Training and prediction time comparison across the three Naïve Bayes variants.

Inference: Multinomial NB is fastest to both train and predict, owing to its simple count-based sufficient statistics; Bernoulli and Gaussian NB are slightly slower due to

the additional per-feature binarization/likelihood computations, but all three remain on the order of milliseconds, confirming Naïve Bayes’s practical suitability for large-scale, low-latency spam filtering.

Additional Tasks

Different train-test splits (Best tuned KNN):

Test Size	Accuracy
0.10	0.9067
0.20	0.9207
0.30	0.9312
0.40	0.9202

Accuracy is fairly stable (0.907–0.931) across all four splits, indicating the model generalizes well and is not overly sensitive to the exact train/test proportion.

Euclidean vs. Manhattan distance ($k = 9$):

Distance	Accuracy
Euclidean	0.9088
Manhattan	0.9034

On the single 80/20 test split Euclidean is marginally ahead, but the 5-fold GridSearchCV result (which averages over many splits) favored Manhattan – the cross-validated result is the more reliable indicator of true generalization performance.

Weighted KNN ($k = 9$):

Weighting	Accuracy
Uniform	0.9088
Distance	0.9164

Distance-weighting improves accuracy by giving closer neighbors more influence over the vote, reducing the impact of borderline/noisy far neighbors – consistent with distance-weighting being selected by both GridSearchCV and RandomizedSearchCV as part of the overall best configuration.

Overfitting/underfitting discussion: Small k (e.g., $k = 1$) fits the training data almost perfectly but is highly sensitive to individual noisy points, leading to higher variance and mild overfitting – visible as lower and less stable accuracy at $k = 1$ and $k = 3$ in Figure 2. Large k averages over many neighbors, smoothing the decision boundary and increasing bias; the plateauing/slight-decline pattern typically observed for very large k (beyond the tested range here) would indicate underfitting, as the neighborhood becomes large enough to mix in points from the opposite class.

12 Discussion

Across every evaluation lens used in this experiment – single-split accuracy/precision/recall/F1, ROC-AUC, 5-fold cross-validation, and precision-recall curves – the tuned KNN classifier consistently outperformed all three Naïve Bayes variants. Among the Naïve Bayes family, Bernoulli NB was clearly the strongest, which is expected given the sparse, largely binary-informative nature of word/character-presence features in spam detection; Multinomial NB underperformed relative to Bernoulli NB here, and Gaussian NB was hurt by the strong non-normality of several features (confirmed in the EDA Q-Q plot). GridSearchCV and RandomizedSearchCV converged on the identical optimal hyperparameter combination (manhattan distance, distance-weighted voting, $k = 9$), demonstrating that RandomizedSearchCV can match exhaustive search quality at roughly $3.3\times$ lower computational cost on this search space. KDTree and BallTree produced identical accuracy (as expected, since both are exact nearest-neighbor algorithms), with BallTree marginally faster in both training and prediction on this dataset's scale. The theoretical complexity table is consistent with the experimental timings: Naïve Bayes's $O(nd)/O(d)$ training/prediction cost is essentially instantaneous, while KNN's tree-based prediction, though sub-linear in theory, carries a larger constant-factor overhead than Naïve Bayes at this dataset size (4,601 samples), which is why the theoretical asymptotic advantage of tree-based KNN prediction does not yet dominate in wall-clock time on this data.

13 Conclusion

This experiment successfully built, tuned, and compared Naïve Bayes (Gaussian, Multinomial, Bernoulli) and KNN classifiers for email spam detection on the Spambase dataset. Bernoulli Naïve Bayes was the best Naïve Bayes variant (Accuracy 0.876, ROC-AUC 0.950), consistent with the dataset's sparse, presence/absence-style features. The optimal KNN configuration, identified identically by GridSearchCV and RandomizedSearchCV, used $k = 9$, Manhattan distance, and distance-weighted voting, achieving the best overall test performance (Accuracy 0.921, ROC-AUC 0.971) and the best 5-fold cross-validation accuracy (0.926). KDTree and BallTree were equivalent in accuracy with BallTree marginally faster. For large-scale or latency-sensitive spam filtering, Naïve Bayes remains attractive due to its near-constant, sub-millisecond training and prediction time; when maximum accuracy is the priority and moderate prediction latency is acceptable, the tuned KNN classifier is preferred based on the results of this experiment.

14 References

1. M. Hopkins, E. Reeber, G. Forman, and J. Suermondt, "Spambase Data Set," UCI Machine Learning Repository, 1999. [Online]. Available: <https://archive.ics.uci.edu/dataset/94/spambase>
2. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.

4. A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, 3rd ed. O'Reilly Media, 2022.